

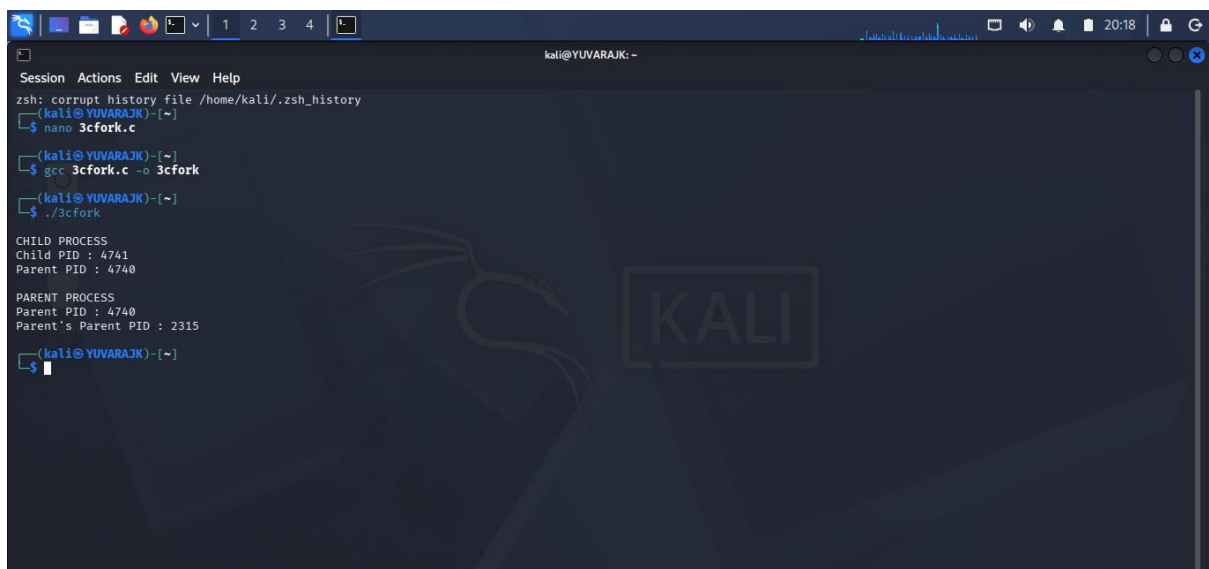
C PROGRAMMING : fork()



```
GNU nano 9.0 3cfork.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0)
    {
        printf("Fork Failed\n");
        exit(1);
    }
    else if(pid == 0)
    {
        printf("\nCHILD PROCESS");
        printf("\nChild PID : %d", getpid());
        printf("\nParent PID : %d\n", getppid());
        exit(0);
    }
    else
    {
        wait(NULL);
        printf("\nPARENT PROCESS");
        printf("\nParent PID : %d", getpid());
        printf("\nParent's Parent PID : %d\n", getppid());
    }
    return 0;
}
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano 3cfork.c
(kali@YUVARAJK)-[~]
$ gcc 3cfork.c -o 3cfork
(kali@YUVARAJK)-[~]
$ ./3cfork

CHILD PROCESS
Child PID : 4741
Parent PID : 4740

PARENT PROCESS
Parent PID : 4740
Parent's Parent PID : 2315
(kali@YUVARAJK)-[~]
$
```

SHELL PROGRAMMING :fork()

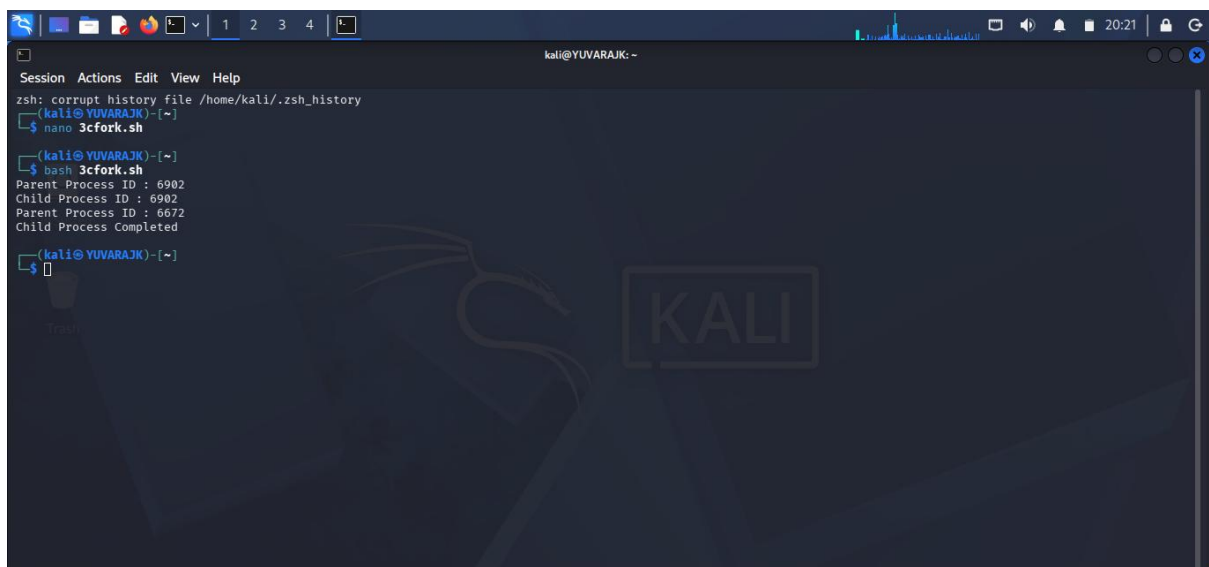


A screenshot of a terminal window on a Kali Linux system. The window title is 'kali@YUVARAJK: ~'. The terminal is running the nano 9.0 text editor, editing a file named '3cfork.sh'. The script content is as follows:

```
#!/bin/bash
echo "Parent Process ID : $$"
(
  echo "Child Process ID : $$"
  echo "Parent Process ID : $PPID"
  exit 0
) &
wait
echo "Child Process Completed"
```

The nano editor interface includes a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. At the bottom, there is a status bar with various keyboard shortcuts like 'H Help', 'X Exit', 'W Write Out', 'R Read File', 'F Where Is', 'N Replace', 'C Cut', 'U Paste', 'T Execute', 'J Justify', 'G Location', 'O Go To Line', 'U Undo', 'R Redo', 'A Set Mark', 'C Copy', 'I To Bracket', 'B Where Was', 'P Previous', and 'F Next'. The background of the terminal features a faint Kali Linux logo.

OUTPUT :

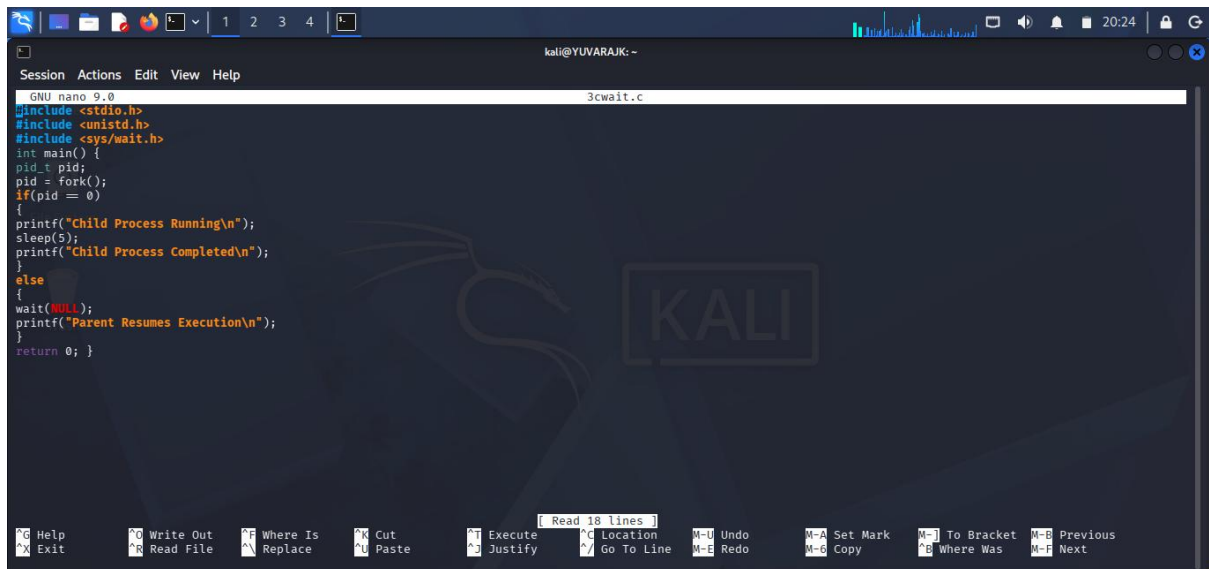


A screenshot of a terminal window on a Kali Linux system, showing the execution of the script. The window title is 'kali@YUVARAJK: ~'. The terminal shows the following sequence of commands and output:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano 3cfork.sh
(kali@YUVARAJK)-[~]
$ bash 3cfork.sh
Parent Process ID : 6902
Child Process ID : 6902
Parent Process ID : 6672
Child Process Completed
(kali@YUVARAJK)-[~]
$
```

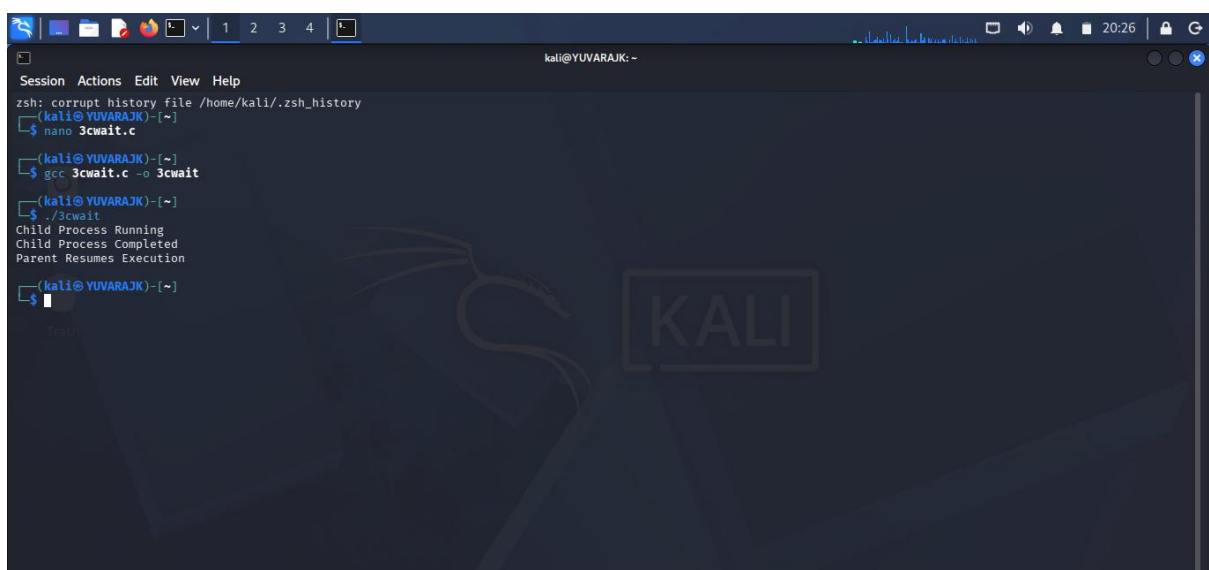
The terminal output displays the parent and child process IDs, confirming that the fork() system call was successful. The background of the terminal features a faint Kali Linux logo.

C PROGRAMMING :wait()



```
GNU nano 9.0 3cwait.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    pid = fork();
    if(pid == 0)
    {
        printf("Child Process Running\n");
        sleep(5);
        printf("Child Process Completed\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Resumes Execution\n");
    }
    return 0; }
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano 3cwait.c
(kali@YUVARAJK)-[~]
$ gcc 3cwait.c -o 3cwait
(kali@YUVARAJK)-[~]
$ ./3cwait
Child Process Running
Child Process Completed
Parent Resumes Execution
(kali@YUVARAJK)-[~]
$
```

SHELL PROGRAMMING :wait()



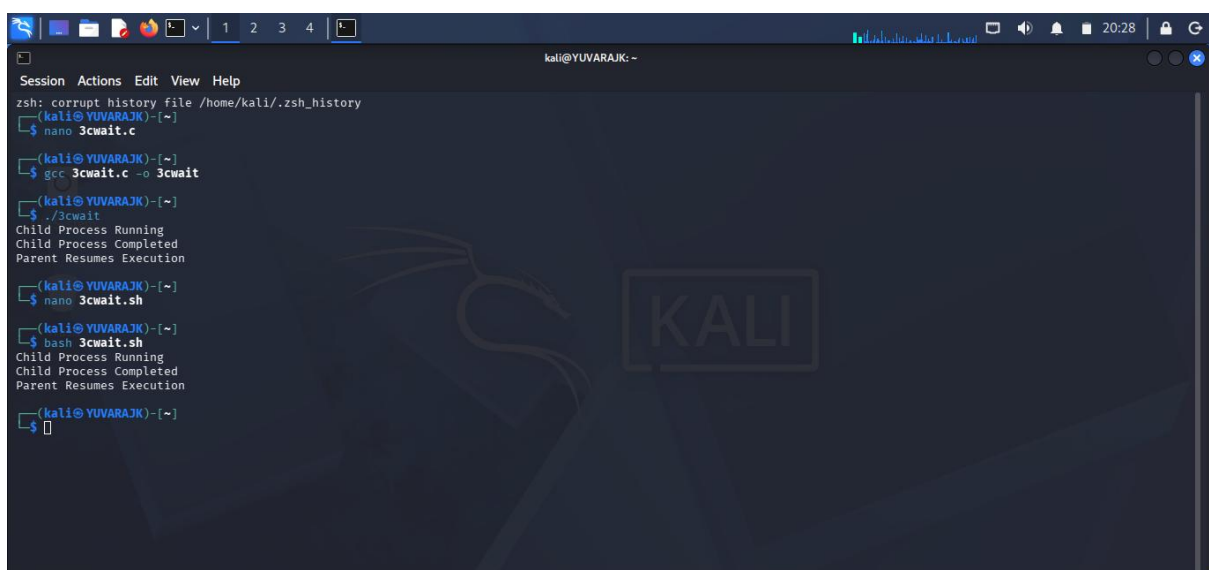
```
GNU nano 9.0 3cwait.sh
#!/bin/bash

(
    echo "Child Process Running"
    sleep 5
    echo "Child Process Completed"
) &

wait

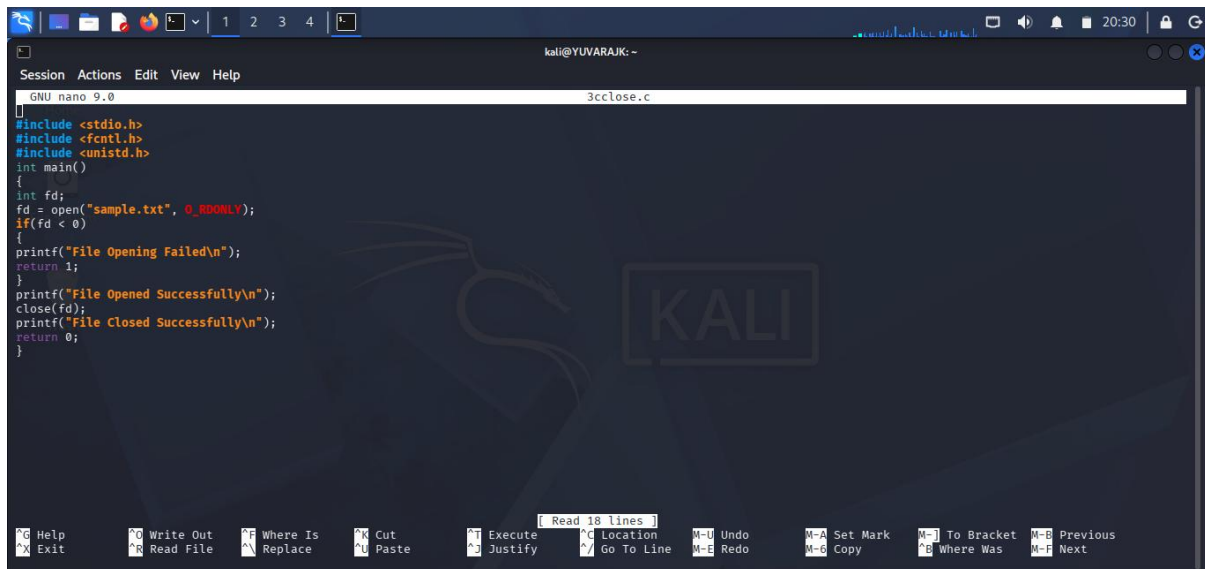
echo "Parent Resumes Execution"
```

OUTPUT :



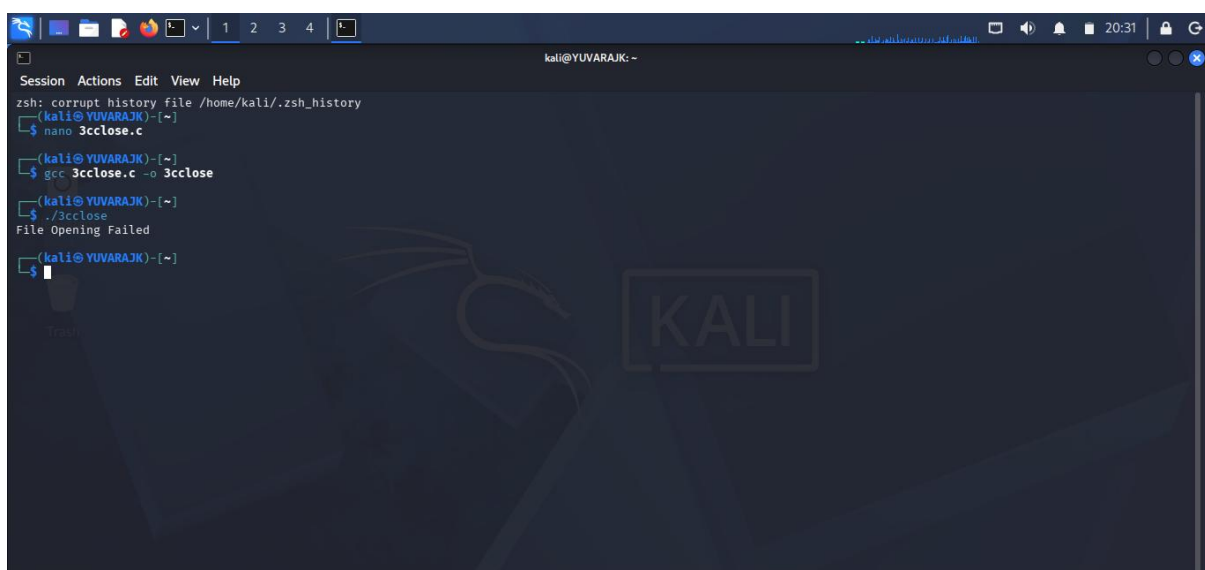
```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano 3cwait.c
(kali@YUVARAJK)-[~]
$ gcc 3cwait.c -o 3cwait
(kali@YUVARAJK)-[~]
$ ./3cwait
Child Process Running
Child Process Completed
Parent Resumes Execution
(kali@YUVARAJK)-[~]
$ nano 3cwait.sh
(kali@YUVARAJK)-[~]
$ bash 3cwait.sh
Child Process Running
Child Process Completed
Parent Resumes Execution
(kali@YUVARAJK)-[~]
$
```

C PROGRAMMING :close()



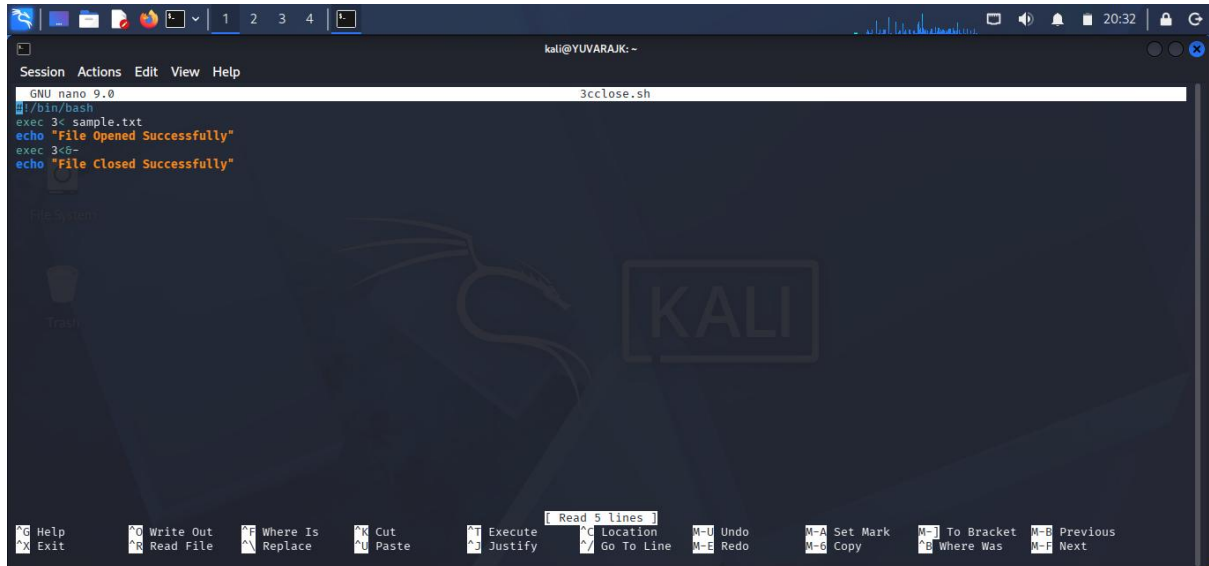
```
GNU nano 9.0 3cclose.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    fd = open("sample.txt", O_RDONLY);
    if(fd < 0)
    {
        printf("File Opening Failed\n");
        return 1;
    }
    printf("File Opened Successfully\n");
    close(fd);
    printf("File Closed Successfully\n");
    return 0;
}
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano 3cclose.c
(kali@YUVARAJK)-[~]
$ gcc 3cclose.c -o 3cclose
(kali@YUVARAJK)-[~]
$ ./3cclose
File Opening Failed
(kali@YUVARAJK)-[~]
$
```

SHELL PROGRAMMING :close()

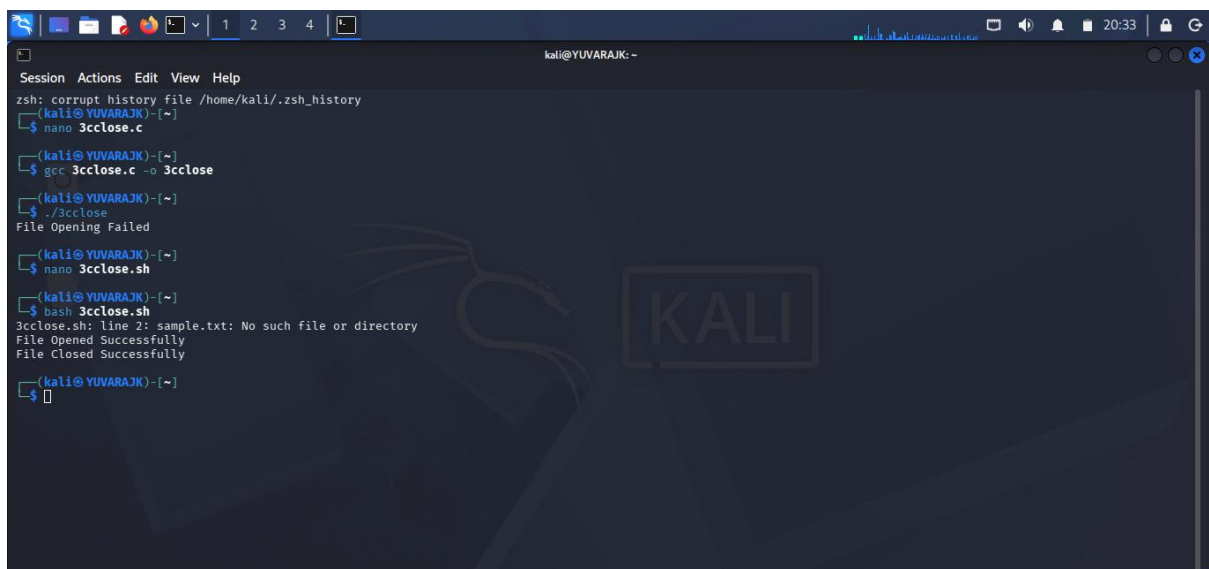


A screenshot of a terminal window on a Kali Linux system. The terminal is running the nano text editor, editing a file named `3cclose.sh`. The file's content is as follows:

```
GNU nano 9.0 3cclose.sh
#!/bin/bash
exec 3< sample.txt
echo "File Opened Successfully"
exec 3<&-
echo "File Closed Successfully"
```

The terminal window has a menu bar with options: Session, Actions, Edit, View, Help. The status bar at the bottom shows various keyboard shortcuts like Help, Exit, Write Out, Read File, etc. The background of the terminal features a faint Kali Linux logo.

OUTPUT :



A screenshot of a terminal window showing the execution of a C program and a shell script. The terminal output is as follows:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@YUVARAJK)-[~]
$ nano 3cclose.c
(kali@YUVARAJK)-[~]
$ gcc 3cclose.c -o 3cclose
(kali@YUVARAJK)-[~]
$ ./3cclose
File Opening Failed
(kali@YUVARAJK)-[~]
$ nano 3cclose.sh
(kali@YUVARAJK)-[~]
$ bash 3cclose.sh
3cclose.sh: line 2: sample.txt: No such file or directory
File Opened Successfully
File Closed Successfully
(kali@YUVARAJK)-[~]
$
```

The terminal window shows the user navigating through the nano editor to create `3cclose.c` and `3cclose.sh`, compiling the C program, and then running both. The shell script execution shows an error for the first attempt (missing file) and success for the second attempt.